

the iterator “actions” and building a new collection. To reiterate: using a while loop to traverse the elements of an iterator and build a new collection (even a new iterator) *does not* qualify as an iterator-to-iterator transformation. Converting an iterator to a more intuitive collection type, manipulating it, and converting back to an iterator is not an iterator-to-iterator transformation. Indeed, converting the iterator argument in `mapPartitions` to a collection object eliminates all the benefits of iterator-to-iterator transformations.

Space and Time Advantages

The primary advantage of using iterator-to-iterator transformations in Spark routines is that their transformations allow Spark to selectively spill data to disk. Conceptually, an iterator-to-iterator transformation means defining a process for evaluating elements one at a time. Thus, Spark can apply that procedure to batches of records rather than reading an entire partition into memory or creating a collection with all of the output records in-memory and then returning it. Consequently, iterator-to-iterator transformations allow Spark to manipulate partitions that are too large to fit in memory on a single executor without out memory errors.

Furthermore, keeping the partition as an iterator allows Spark to use disk space more selectively. Rather than spilling an entire partition when it doesn’t fit in memory, the iterator-to-iterator transformation allows Spark to spill only those records that do not fit in memory, thereby saving disk I/O and the cost of recomputation. Lastly, using methods defined on iterators avoids defining intermediary data structures. Reducing the number of large intermediate data structures is a way to avoid unnecessary object creation, which can slow down garbage collection as we talked about in [“Minimizing Object Creation”](#).

NOTE

Unfortunately the Spark Streaming `mapPartitions` API is one of relatively few places where the Scala API decisively outperforms its Java counterpart. Prior to Spark 1.6, `mapPartitions` in Spark Streaming was defined on objects of type `Java Iterable` rather than `Java Iterator` and thus automatically reads the entire collection into memory. In the Spark Core, the Java API still uses `Iterable` rather than iterators as the grouped result of `groupByKey`, thus eliminating the possibility of using an iterator-to-iterator transformation after a `groupByKey` call.

An Example

For all their advantages, iterators can be a much harder abstraction to conceptualize and use than collection types such as arrays and hash maps, with which users may be more familiar from other languages. Here we provide an example of a complicated `mapPartitions` routine, which given a sorted RDD of `(value, columnIndex, count)` tuples and a list of rank statistics on this partition, returns the `(value, columnIndex)` pairs that represent ranks statistics, shown in [Example 5-10](#). This method is part of the optimal solution to the “Goldilocks problem,” which is presented in full in [“Goldilocks Version 4: Reduce to Distinct on Each Partition”](#) and introduced in [“The Goldilocks Example”](#).

Example 5-10. Example mapPartitions

```
private def findTargetRanksIteratively(
  sortedAggregatedValueColumnPairs : RDD[((Double, Int),
Long)],
  ranksLocations : Array[(Int, List[(Int, Long)])]):
RDD[(Int, Double)] = {

sortedAggregatedValueColumnPairs.mapPartitionsWithIndex((partition
Index : Int,
  aggregatedValueColumnPairs : Iterator[((Double, Int),
```

```

Long)) => {
    val targetsInThisPart: List[(Int, Long)] =
    ranksLocations(partitionIndex)._2
    if (targetsInThisPart.nonEmpty) {
        FindTargetsSubRoutine.asIteratorToIteratorTransformation(
            aggregatedValueColumnPairs,
            targetsInThisPart)
    } else {
        Iterator.empty
    }
}
}
}

```

This routine is a good example of a place where we are likely to see performance gains from an iterator-to-iterator transformation, since it is a complicated routine performed on partitions that we anticipate will be too large to fit in memory. However, it is an instance where using iterators is, from a design perspective, a non-obvious choice because we have to keep a map of running totals with the number of elements for each column we have seen so far. A more straightforward way to design this routine would be as follows: loop through the iterator, store the running totals in a `HashMap`, and build a new collection of the elements we want to keep using an array buffer—then convert the array buffer to an iterator, shown in [Example 5-11](#).

Example 5-11. MapPartitions example without an iterator-to-iterator transformation

```

def withArrayBuffer(valueColumnPairsIter : Iterator[((Double,
Int), Long)],
    targetsInThisPart: List[(Int, Long)] ): Iterator[(Int,
Double)] = {

    val columnsRelativeIndex: Predef.Map[Int, List[Long]] =
        targetsInThisPart.groupBy(_._1).mapValues(_._2)

    // The column indices of the pairs that are desired rank
    // statistics that live in
    // this partition.
    val columnsInThisPart: List[Int] =

```

```

targetsInThisPart.map(_._1).distinct

    // A HashMap with the running totals of each column index. As
    // we loop through
    // the iterator, we will update the hashmap as we see elements
    // of each
    // column index.
    val runningTotals : mutable.HashMap[Int, Long]= new
mutable.HashMap()
    runningTotals += columnsInThisPart.map(columnIndex =>
(columnIndex, 0L)).toMap

    //we use an array buffer to build the resulting iterator
    val result: ArrayBuffer[(Int, Double)] =
    new scala.collection.mutable.ArrayBuffer()

    valueColumnPairsIter.foreach {
        case ((value, colIndex), count) =>

            if (columnsInThisPart contains colIndex) {

                val total = runningTotals(colIndex)
                //the ranks that are contained by this element of the
                input iterator.
                //get by filtering the
                val ranksPresent = columnsRelativeIndex(colIndex)
                .filter(index => (index <= count + total) && (index
> total))
                ranksPresent.foreach(r => result += ((colIndex,
value)))
                //update the running totals.
                runningTotals.update(colIndex, total + count)
            }
        }
    }
    //convert
    result.toIterator
}

```

At first this looks like an okay solution since we are estimating that the number of elements we are returning is small, and because array buffers are usually a relatively performant way to build up Scala collections. However, if the input data is very large relative to the cluster size, we still see out-of-memory errors and failures in this step. A more efficient solution would be to use an iterator-to-iterator transformation. We can

convert this subroutine to an iterator-to-iterator transformation although our routine is not parallelizable (it requires keeping a list of running totals). We can do this because the subroutine we need can be completed on one element of the iterator without any information about the other elements. The final solution uses the `filter` function of iterators—to eliminate any elements that are not in the final data—and a `flatMap` to build the new iterator of elements in the resulting partitions, as shown in [Example 5-12](#).

Example 5-12. MapPartitions with iterator-to-iterator transformations

```
def asIteratorToIteratorTransformation(
  valueColumnPairsIter : Iterator[((Double, Int), Long)],
  targetsInThisPart: List[(Int, Long)] ): Iterator[(Int,
Double)] = {

  val columnsRelativeIndex =
    targetsInThisPart.groupBy(_._1).mapValues(_._2.map(_._2))
  val columnsInThisPart = targetsInThisPart.map(_._1).distinct

  val runningTotals : mutable.HashMap[Int, Long]= new
  mutable.HashMap()
  runningTotals += columnsInThisPart.map(columnIndex =>
    (columnIndex, 0L)).toMap

  //filter out the pairs that don't have a column index that is
  in this part
  val pairsWithRanksInThisPart = valueColumnPairsIter.filter{
    case (((value, colIndex), count)) =>
      columnsInThisPart contains colIndex
  }

  // map the valueColumn pairs to a list of (colIndex, value)
  pairs that correspond
  // to one of the desired rank statistics on this partition.
  pairsWithRanksInThisPart.flatMap{

    case (((value, colIndex), count)) =>

      val total = runningTotals(colIndex)
      val ranksPresent: List[Long] =
        columnsRelativeIndex(colIndex)
                                                .filter(index => (index
<= count + total)
```

```

                                && (index > total))

    val nextElems: Iterator[(Int, Double)] =
        ranksPresent.map(r => (colIndex, value)).toIterator

    //update the running totals
    runningTotals.update(colIndex, total + count)
    nextElems
}
}

```

This approach allows the function to spill to disk selectively by working with each element in the iterator one at a time. This implementation saves space by incrementally building the result rather than storing the new collection type in memory as an array buffer. It saves a penny on garbage collection by not creating the array buffer as an intermediate step.

TIP

If you are using an `ArrayBuffer` to build a new collection for `mapPartitions`, it is always possible (and likely more performant) to use a `map` or `flatMap` on the iterator to incrementally add new elements.

Set Operations

Spark has a variety of set-like operations, some of which are expensive and some of which have different behavior than the mathematical definitions of the equivalent operations. In this section we hope to explain how to use these operations safely and effectively.

Since RDDs aren't distinct, they mainly differ from mathematical set operations in how they handle duplicates. For example, `union` merely combines its arguments, so the result of `union` will always have the size of both RDDs combined. `intersection` and `subtract` are defined

similarly to their set-theoretic counterparts, but since the input RDDs (unlikely mathematical sets) can have duplicates the results may be unexpected. Subtracting will remove all of the elements in the first RDD that have a key present in the second RDD. Thus it is possible that by subtracting, the result will be smaller than the size of the first RDD minus the size of the second, breaking one of the laws of set theory.

For example, the simple unit test in [Example 5-13](#) will pass.

Example 5-13. Subtract example

```
val a = Array(1, 2, 3, 4, 4, 4, 4)
val b = Array(3, 4)
val rddA = sc.parallelize(a)
val rddB = sc.parallelize(b)
val rddC = rddA.subtract(rddB)
assert(rddC.count() < rddA.count() - rddB.count())
```

In Spark, `intersection` co-groups the argument RDDs using their values as keys and filters out those elements that don't appear in both. Consequently the result of RDD `intersection` contains no duplicates. Although this is the expected behavior for `intersection`, using several set operations on RDDs containing duplicates can lead to unexpected behavior. The union of the two RDDs in [Example 5-13](#) is an RDD containing two elements, 1 and 2. Thus, as the unit test in [Example 5-14](#) demonstrates, we cannot always “re-create” `rddA` as the union of the intersection and the subtraction.¹

Example 5-14. Intersection example

```
val a = Array(1, 2, 3, 4, 4, 4, 4)
val b = Array(3, 4)
val rddA = sc.parallelize(a)
val rddB = sc.parallelize(b)
val intersection = rddA.intersection(rddB)
val subtraction = rddA.subtract(rddB)
val union = intersection.union(subtraction)
```

```
assert(!rddA.collect().sorted.sameElements(union.collect().sorted)
)
```

TIP

To make an RDD more like a set, you can use `distinct` prior to computing any set operations. However, calling `distinct` will cause a shuffle, and if the data is not already partitioned this can be expensive.

Reducing Setup Overhead

Some operations require setup work per-worker or per-partition, like creating a database connection or setting up a random number generator. For transformations you can use `mapPartitions`, do the setup work per partition in the map function, and then perform your desired transformation on the iterator for the partition. We will illustrate doing this with a pseudorandom number generator in [Example 5-15](#).

Example 5-15. Create one random number generator per partition using broadcast variable

```
rdd.mapPartitions{itr =>
  // Only create once RNG per partitions
  val r = new Random()
  itr.filter(x => r.nextInt(10) == 0)
}
```

TIP

It is important to remember to use an iterator-to-iterator transformation to allow spilling to disk selectively, as discussed in [“Iterator-to-Iterator Transformations with mapPartitions”](#).

Beyond using this pattern to reduce setup overhead in transformations, another common pattern is to create a connection inside of an action to save the data. If your work is writing out the data you can use the same pattern as with `mapPartitions` except with `foreachPartition`.

If the setup work can be serialized, a broadcast variable can distribute the object that we cover next. If the setup work can't be serialized, a broadcast variable with a `transient lazy val` can be used as well. See [Example 5-17](#) in the next section.

Shared Variables

Spark has two types of shared variables—broadcast variables and accumulators—each of which can only be written in one context (driver or worker, respectively) and read in the other. Broadcast variables can be written in the driver program and read on the executors, whereas accumulators are written onto the executors and read on the driver.

Broadcast Variables

Broadcast variables give us a way to take a local value on the driver and distribute a read-only copy to each machine rather than shipping a new copy with each task. Broadcast variables might not seem especially useful, since we can just capture a local variable in our closure to transfer data from the driver to the workers; however, the savings of only sending one copy per machine versus sending one copy per task can make a huge difference, especially when the same broadcast variable is used in additional transformations. Two common examples of using broadcast variables are a) broadcasting a small table to join against and b) broadcasting a machine learning model to be able to run the predictions on our data.

Creating a broadcast variable is done by calling `broadcast` on the `SparkContext`. This distributes the value to the workers and gives us back a wrapper that allows us to access the value on the workers by calling `value`, as shown in Examples 5-16 and 5-17. If a broadcast variable is created with a variable input, the input should not be modified after the variable has been created since existing workers will not see the updates and new workers may see the new value.

Example 5-16. Sample broadcast of a hashset of invalid panda locations to filter out

```
val invalid = HashSet() ++ invalidPandas
val invalidBroadcast = sc.broadcast(invalid)
input.filter{panda =>
!invalidBroadcast.value.contains(panda.id)}
```

Example 5-17. Create one random number generator per worker

```
class LazyPrng {
  @transient lazy val r = new Random()
}
def customSampleBroadcast[T: ClassTag](sc: SparkContext, rdd:
RDD[T]): RDD[T] = {
  val bcastprng = sc.broadcast(new LazyPrng())
  rdd.filter(x => bcastprng.value.r.nextInt(10) == 0)
}
```

WARNING

The value for a broadcast variable must be a local, serializable value: no RDDs or other distributed data structures.

Internally, Spark uses broadcast variables for the Hadoop job configuration objects and large blocks of Python code for UDFs. If a broadcast variable is no longer needed, you can explicitly remove it by calling `unpersist()` on the broadcast variable.

Accumulators

Accumulators are the second type of Spark's shared variables, allowing us to collect by-product information from a transformation or action on the workers and then bring the result back to the driver. With Spark's execution model, Spark adds to accumulators only once the computation has been triggered (e.g., by an action). If the computation happens multiple times, Spark will update the accumulator each time. This multiple counting can be desirable for process-level information, like computing the entire time spent parsing records. However, it can be disastrous for data-related information like counting the number of invalid records.

NOTE

Spark accumulators have had an API update for 2.0—these examples are updated for the 2.X API, although 1.X examples are still available [in the examples repo](#).

WARNING

Accumulators can be unpredictable. In their current state, they are best used where potential multiple counting is the desired behavior.

Accumulators have a number of built-in types that make it easy to create an accumulator for common use cases. Accumulators are not intended for collecting large amounts of information, so if you find yourself adding a large number of elements to a collection or appending to a string you may wish to consider a separate action instead of an accumulator. The default operation for numeric accumulators is the + operation, so we could use this to sum the fuzzyness of all of the pandas as shown in [Example 5-18](#).